DV2597 - Lab 2

Christoffer Bohman - chbh22@student.bth.se

March 19, 2026

1 Odd-Even Sort

1.1 Implementation

Quick sort is an algorithm that depends on the idea of segmenting a list of numbers based on pivot that is decided by a heuristic function that may be adjusted according to the needs and assumptions of a program. An optimised parallellisation of this algorithm could require multiple techniques and mechanics.

In my specific case, two main principles were used:

- Thread Separation

  - As a naive implementation of a parallellised quick sort algorithm could cause different issues such as data races, a work separation strategy needs to be created. In my case I decided to give each thread their own area of responsibilty by only allowing one thread to access one of the halves that have been partitioned by the heuristic function. This consequently makes each partition single-threaded but the entire list multi-threaded, resolving any potential conflicts.

- Work & Thread Pooling

  - An optimisation that had to be implemented for the parallell variant to be faster than the sequential one, was pooling. What this does is create all required threads from the start, minimising the overhead of thread creation and maximising reuse where possible.

  The work pooling aspect works by creating an array of double-ended queues (henceforth known as DE-queues) with work with one array index per thread. After a thread has successfully partitioned a segment, both the left and right hand sides of that segment get pushed up to that threads DE-queue. The thread then pops the next piece of work from the front of its own DE-queue. If a threads DE-queue is empty, it will actively try to steal work from other threads by taking from the back end of their DE-queues, resulting in a better management and a more proper distribution of work.

  For further optimisation, the overhead of work pooling has been taken into account. The cut-off point for the thread to simply continue without pushing up to a queue is therefore set to 10000.

1.2 Measurements

Table 1: Measurements - Quick Sort

Number of Threads	Average Time (s)
Sequential	16.10266
Parallell (1 Thread)	17.93206
Parallell (2 Threads)	9.63332
Parallell (4 Threads)	5.338844
Parallell (8 Threads)	3.70554
Parallell (16 Threads)	2.833684
Parallell (32 Threads)	2.667642

Table 1: Average execution times of the sequential and parallell variants of quick sort respectively.

2 Gaussian Elimination

2.1 Implementation

Gaussian elimination is a mathematical method that can help find the solutions to certain linear equations. It works by taking a given matrix and systematically dividing each row as well as eliminating each column which consequently zeroes out each element under the matrix diagonal.

A parallellisation of this algorithm requires the understanding of critical zones and work distribution between threads as to not to cause data races and data corruption. For that reason, my implementation makes use of both barriers and single threading in affected areas.

First of all, we replicate the division step by assigning elements to each thread according to their thread ID and dividing them with the element on the diagonal. Since each element is reliant on the diagonal element, these need to be done first. After this, in order to ensure no data races happen, only the thread with ID 0 changes the value of the result and the diagonal diagonal element.

After this, the elimination step occurs. Like the previous step, each thread gets assigned elements based on the thread ID. The elements then adjust the rows below the current row using the current rows values and also setting the column below the diagonal to 0.0. Both steps then repeat for all rows.

2.2 Measurements

Table 2: Measurements - Gauss Elimination

Number of Threads	Average Time (s)
Sequential	10.93124
Parallell (1 Thread)	12.33312
Parallell (2 Threads)	6.700142
Parallell (4 Threads)	3.875878
Parallell (8 Threads)	2.775094
Parallell (16 Threads)	2.386
Parallell (32 Threads)	1.915616

Table 2: Average execution times of the sequential and parallell variants of gauss elimination respectively.